

HighGround — context for the landing page

Everything a teammate needs to build the landing page without reading the codebase. Numbers in this file are copied from the model output files (`data/out/web/proof.json`, `runs.json`, `hospitals.json`, `streets/index.json`, `graph/meta.json`) on Sat 10 Oct 2026. **If you quote a number, quote it exactly as written here, with its caveat.** Never round a validation number up or drop its caveat: honesty is part of the pitch.

1. One line, and the pitch

HighGround shows Chennai residents where floodwater will go tonight, street by street, and what to do about it.

- Will my street flood tonight? How deep, and when?
- Where can I park my car on dry ground?
- Which way can I drive there without crossing water, and by when must I leave?
- Which hospitals get cut off?
- Email me if this changes.

Positioning (use this language): Chennai already has CFLOWS, a government flood warning system built for officials. HighGround is for the residents who still park their cars on the Velachery flyover every storm because nobody tells them where the water will go. We do not claim a better model than CFLOWS; we claim a tested one, built for citizens.

Positioning update (Sun 11 Oct): the state's **Chennai Flood Monitor** (public, since Oct 2025) claims street-level inundation forecasts for some areas, and **TN-Alert** (state app, 5 lakh+ installs) sends official alerts. So never say HighGround is the only street-level forecast. Say what it adds: *Official systems tell you a flood is coming. HighGround tells you what to do on your street: where to move your car and by when, which way to go without crossing water, which hospitals you can still reach, and it keeps working when the power and the network go down.*

Tagline ideas (pick or rewrite):

- "Where the water goes tonight, street by street."
 - "Know before the water does."
 - "Every monsoon, Chennai parks its cars on flyovers. Now you know why, and where."
-

2. The event

- **Environmental Hacks**, event 02 of the **Bharat Builds Tour** (WeMakeDevs × AWS), **Heat and Water** track. Hybrid, Oct 8–11, 2026.
 - **Submission closes Sun 11 Oct 2026, 20:00 IST** (confirm on the wemakedevs.org/aws/env form).
 - Judged on: **idea and impact**, **built on AWS** (and shown in the video), **design and usability**, **execution**, and a **3-minute demo video**. Blog on AWS Builder Center for the blog prize.
 - Repo: https://github.com/Divue/high_ground · Live URL: **TBD after AWS deploy** · Video: **TBD**.
-

3. What a resident sees (the product)

The whole app is one full-screen 3D night-time map of Chennai with floodwater on it. Screens:

Screen	What it does
Tonight (hero)	Type your street → the camera flies across the city to it → the night plays from 6 PM and the water rises hour by hour → an answer card: depth at its worst in plain words ("Waist-deep", with the number in cm), when it gets too deep for scooters, a picture of an adult / scooter / car with the water level, "Move your car to X by 2 AM", the nearest dry parking (mostly flyovers), an amber dry route that draws itself, and "Email me if this changes".
Replay a storm	Tonight's live forecast is usually dry, so the app can replay a real storm as if it were tonight: Dec 2015 floods, Cyclone Michaung 2023, Cyclone Fengal 2024.
Watch the whole storm	A ~30-second time-lapse of a replay on its real dates, with captions that come from the model files ("Tue 1:30 PM: 43 mm fell in this one hour...", "Chembarambakkam reservoir starts releasing water..."). Pause, resume, drag, watch again.
What if	Drag rainfall from 50 to 400 mm in a day (mean or high tide); the city floods and drains; "% of the city too deep for scooters" and "hospitals cars cannot reach".
Hospitals	46 major hospitals; which are cut off from the main road network at each storm size, and the dry route to each one that is not.
Proof	"Did the model get 2015 right?" — a swipe split-screen: the model's 2015 flood on the left, streets residents reported flooded on the right, and the honest result (see section 6).
About the model	Plain-language limits and data credits.
Take me to dry ground (new)	On foot, by two-wheeler or by car, to dry parking, a hospital, high ground or a saved place. The route avoids streets the model expects to flood at the hour you would reach them, closes subways in rain and keeps to one-way streets. Step-by-step directions; Start follows your GPS with one big instruction and voice; Preview the drive plays it.
Save for offline (new)	One tap before the storm keeps the app, the map, your street (and two more places) for every storm, and the road network on the phone. When the power and the towers go, it still opens and still plans a way out. Battery-saver map with streets coloured by depth.
Help numbers + My flood plan (new)	Tap-to-call 112, 1913, 1070, 1077, 108, 101 and the power line for fallen wires; send your location by SMS/WhatsApp. The flood plan is an image and text to send to family; replays are stamped "Practice plan, not a forecast".
Ask HighGround	An AI assistant that only answers from the model's numbers (Amazon Bedrock + Strands Agents) and tells people in danger to call 112 first.

Interaction details worth showing on the landing page: tap any street for its depth; a play button that plays the night; map controls (zoom, tilt, 3D/flat, spin, back to my street); flowing water that runs downhill; rain; a "Play animations" offer for devices that ask for reduced motion.

4. How it works (three layers)

OFFLINE (Python on a laptop) Copernicus 30 m elevation (AWS Open Data) ESA WorldCover land cover (AWS Open Data) OpenStreetMap (buildings, roads, drains, hospitals, flyovers) Rain scenarios + 3 real storms → terrain conditioning → 2D flood model (16 storms) → validation vs 2015 reports → street / route / hospital / texture files → S3 + CloudFront	ONLINE (AWS) EventBridge Scheduler (every 3 h) → Lambda forecast-check: Open-Meteo rain for 4 points → pick the 2 nearest storms → current.json on S3 → risk changed? Bedrock writes 2 plain sentences → SNS email API Gateway → Lambda subscribe (DynamoDB + SNS filter policy) API Gateway → Lambda assistant (Strands Agents on Bedrock)	BROWSER MapLibre GL 3D city + terrain three.js water layer: hourly depth textures blended on the GPU, flowing water, rain A* routing over 206k road edges, hour by hour All numbers from model files
--	---	--

A clean architecture diagram is in `docs/landing-assets/10_architecture.png` (also `docs/architecture.png`).

The flood model, in plain words

1. **Ground:** the satellite elevation model (Copernicus GLO-30, 30 m) includes rooftops and trees. We removed 371,679 OpenStreetMap buildings, tree canopy (ESA WorldCover), bridges and other tall objects, filled the gaps from surrounding ground, carved rivers, canals and drains into it, and put buildings back as walls that water flows around.
2. **Water:** a 2D "shallow water" flood model (the local-inertial scheme, Bates et al. 2010) written in Python/numba runs on ~1.2 million 30 m cells covering Greater Chennai. Rain falls on every cell, drains and soil take some away, water flows downhill street by street, and leaves into the sea at the tide level. Every run conserves water exactly (mass error 0.000%).
3. **Storms:** 12 design storms (50, 100, 150, 200, 300, 400 mm in a day, each at mean and high tide) and 3 real ones: 1–2 Dec 2015 (374 mm, run with and without the Chembarambakkam reservoir release, modelled from the CAG audit timeline, peak 29,000 cusecs), Cyclone Michaung 2023 (415 mm), Cyclone Fengal 2024 (114 mm). 16 runs in total.
4. **Per street:** each of 168,156 street pieces gets an hour-by-hour depth, its worst depth and the hour it gets too deep for scooters (15 cm). 168 dry-parking candidates (122 of them flyovers), a 206,223-edge road graph for dry routes, 46 hospitals.
5. **Cross-check:** a second, independent flood model (ANUGA, Geoscience Australia) on a 103,159-triangle mesh over Velachery and Pallikaranai.
6. **Live:** every 3 hours AWS reads the rain forecast, picks the nearest precomputed storms, and emails subscribers whose street's risk changed.

The browser never runs the flood model: everything is precomputed, so the 3D map stays smooth (~57 fps measured on an integrated-GPU laptop with the water flowing).

5. Tech stack

Layer	Tools (versions as installed)
Frontend	Vite 8.3.4, React 19.3, TypeScript, MapLibre GL JS 6.13.0 (3D terrain, extruded buildings, colour-relief, hillshade), three.js 0.186.1 (custom water layer, GLSL shaders), PMTiles 4.5.0 + Protomaps basemaps 5.7.2 (dark basemap from one file on S3), Web Workers (image decoding), Canvas 2D (rain)
Model pipeline	Python 3.11, numba (solver), NumPy, SciPy, rasterio/GDAL, GeoPandas, pyproj, scikit-image, pandas; ANUGA 4.0.1 (cross-check)
AWS	Amazon S3 + Amazon CloudFront (tiles, model outputs), AWS Amplify Hosting (site), AWS Lambda (Python 3.12: subscribe, forecast-check, assistant, geocode), Amazon API Gateway (HTTP API), Amazon DynamoDB (subscribers), Amazon SNS (alert email with per-street filter policy), Amazon EventBridge Scheduler (every 3 h), Amazon Bedrock (Claude: alert text + assistant), Strands Agents SDK (AWS open source, the assistant), Amazon Location Service (address search, biased to Chennai), AWS SAM (whole stack in one template), AWS Open Data Registry (Copernicus DEM and ESA WorldCover read straight from S3)
Offline + navigation	vite-plugin-pwa + Workbox (service worker), Cache API, self-hosted fonts and map glyphs, Web Workers (routing), Geolocation, Screen Wake Lock, Web Speech, Web Share

Layer	Tools (versions as installed)
Testing / tooling	Playwright (end-to-end tests, frame-perfect demo footage with a virtual clock), moto (AWS mocks for the Lambda tests), ffmpeg
Other data/services	OpenStreetMap, Open-Meteo forecast API, ERA5 (storm timing via Open-Meteo), IMD rainfall totals, OpenCity datasets (2015 crowd-sourced flood reports, GCC flood hazard zones, NRSC 2015 inundation extent, GCC ward map)

6. Key numbers you may quote (with their caveats)

Validation (the honest headline)

- Test: on wards **never used for tuning**, pick a street residents reported flooded in Dec 2015 and one they did not. How often does the model put more water on the reported one?
- **Model: 53%** (95% range 51–55%, resampling whole wards). **Coin toss: 50%**. Low ground alone: 47%. "How close is the nearest canal": **56%** (a little better than the model).
- Verdict, word for word: **"slightly better than chance"**. Treat a street's number as a guide to how the area behaves, not a promise. *Quote this together; never quote 53% alone.*
- 3,200 reported street segments in the held-out wards.
- The Chembarambakkam reservoir release matters on river-side streets: ranking 51% without it, 53% with it.
- ANUGA cross-check (Velachery, 200 mm storm, first 14 hours): the two models agree on **73%** of the flooded area (critical success index), depth correlation 0.90. *This checks the maths, not the terrain.*

Storm outcomes (share of the city's land under ≥ 15 cm, "too deep for scooters")

Storm	Rain	Land ≥ 15 cm	Hospitals cut off (of 46)
100 mm design storm	100 mm / 24 h	2.4%	0
200 mm design storm	200 mm / 24 h	17%	9
300 mm design storm	300 mm / 24 h	25%	15
400 mm design storm	400 mm / 24 h	30%	16
Dec 2015 (with release)	374 mm	29%	16
Cyclone Michaung 2023	415 mm	23%	15
Cyclone Fengal 2024	114 mm	3.6%	0

Scale

- ~1.2 million 30 m cells; 371,679 buildings removed from the elevation model; 168,156 street pieces; 206,223 road edges; 168 dry-parking candidates (122 flyovers); 46 hospitals; 16 storm runs; mass error 0.000% on every run.

7. Claims to make, and claims to avoid

Make: street-by-street answers in plain words; tested honestly against 2015 on held-out wards; an independent second model agrees on 73% of the flooded area; built for residents, not officials; AWS-native, open data, open source tools; alerts by email (SMS coming: Indian SMS needs DLT registration).

Avoid: "accurate", "predicts exactly", "official", "better than CFLOWS", "real-time flood model" (the model is precomputed; the forecast check is live), "AI predicts floods" (the AI only explains the model's numbers), any validation number without its caveat, "calibrated" (drain capacity is a stated assumption, 10 mm/h), SMS alerts as working.

8. Limits (must appear somewhere on the page)

- Terrain is 30 m satellite elevation with buildings and trees removed by approximation: kerbs, gates and small dips are invisible to it.
- Drains are one uniform capacity (10 mm/h), an assumption — not a drain network.
- The model does not reproduce GCC's finding that Velachery floods far more than T. Nagar; its flooding follows small hollows in the 30 m terrain.
- The 2015 reservoir release is modelled from the CAG audit timeline; other tanks are not included.
- No storm surge; rain falls evenly across the city in each scenario.
- **Not an official warning. Follow GCC and IMD advisories. In danger, call 112.**

9. Design language (match the app)

Concept: Chennai at 2 AM, mid-storm. The 3D city is the hero; the water is the brightest thing on screen; everything else is quiet.

Token	Hex	Use
Night background	#050D12	page background
Land	#0A161C	map land, dark surfaces
Panel	rgba(8, 19, 25, 0.86) (solid #0A171D)	cards, with a 1px border rgba(214, 224, 228, 0.10), radius 14px
Text	#D6E0E4 (dim: #8FA2AA)	all text
Shallow water	#86E6EC	water ramp start, accents for water
Deep water	#1E7BC6	water ramp end
Sodium amber	#F2A541	only safe places, parking pins, the safe route
Emergency red	#E5484D	only the "call 112" line
Original spec tokens	storm sky #14303D, wet asphalt #3E5560	older, slightly lighter teal look

- **Type:** **Anek Latin** (variable width; headlines and big numbers, it widens and gets bolder as depth grows) and **Hind** (body). Both on Google Fonts.
- **Voice:** sentence case everywhere; plain words ("too deep for scooters", "waist-deep", "move your car to X by 2 AM"), never jargon (no "AUC", "p90", "scenario", "inundation"). Numbers always come with a plain phrase.
- **Rules from the spec:** amber means safety and nothing else; red only for 112; no all-caps labels; no arrows on buttons; no glassmorphism card grids or gradient blobs; no hover animation on everything; one orchestrated moment (the flight in).
- **Wordmark:** "HighGround" set in Anek Latin, weight ~650, slightly wide. There is no logo image.

10. Assets ready to use (docs/landing-assets/)

File	What it shows
01_hero_street.png	The hero: a street in Velachery during Cyclone Michaung, answer card, amber route, controls
02_whatif_city.png	What if at 200 mm: the whole city as a glowing flood map
03_proof.png	Proof: "Did the model get 2015 right?" split screen
04_hospitals.png	Hospitals cut off at 300 mm
05_timelapse_2015_peak.png	"Watch the whole storm", Dec 2015 near its worst hour
06_search_flight.png	Mid-flight to a searched street ("Going to Arumugam Road")
07_opening_title.png	The opening: "Chennai, 6 PM. Cyclone Michaung, 2023, replayed as if it were tonight."
08_street_3d_rain.png	Street-level 3D view with rain and coloured buildings
09_motion_offer.png	The "Play animations" offer for reduced-motion devices
10_architecture.png	Architecture diagram with every AWS service

More: [review/p5-dev/](#) (many screenshots), and smooth 1080p demo clips will be in [review/p5/footage/](#) (opening.mp4, search.mp4, timelapse.mp4, whatif.mp4, proof.mp4) — ask before using if they are not there yet. Screenshots are 1440×900 or 1280×800 PNG.

11. Suggested landing-page sections (optional; your call)

1. **Hero:** full-bleed 3D night city with water (01_hero_street.png or a looping clip), the one-line pitch, buttons "Open the map" and "Watch the 3-minute demo".
2. **The problem:** cars on the Velachery flyover; CFLOWS is for officials; residents get WhatsApp forwards.
3. **What it answers:** the five questions (section 1) with small screenshots.
4. **See a real storm:** Michaung / 2015 / Fengal replays and the time-lapse.
5. **Is it right? (Proof):** the honest result in plain words with the coin-toss comparison and the ANUGA agreement — and the caveat.
6. **Built on AWS:** the architecture diagram and the list of services (section 5).
7. **Limits and credits:** section 8 and the attribution lines (section 12).
8. **Team, repo, blog, video links.**

12. Attribution lines (required wherever the data or map appears)

- Elevation: Copernicus DEM GLO-30, © DLR e.V. 2010–2014 and © Airbus Defence and Space GmbH 2014–2018, provided under COPERNICUS by the European Union and ESA; all rights reserved. Read from the AWS Open Data registry.
- Land cover: ESA WorldCover 2021 (CC BY 4.0), © ESA WorldCover project / contains modified Copernicus Sentinel data (2021), from the AWS Open Data registry.
- Map data: © OpenStreetMap contributors; basemap by Protomaps.
- Rainfall forecasts and storm timing: Open-Meteo (CC BY 4.0), including ERA5 reanalysis (Copernicus Climate Change Service).
- 2015 flood reports: OpenCity, osm-in/flood-map contributors. Flood hazard zones and ward map: Greater Chennai Corporation via OpenCity. 2015 inundation extent: NRSC via OpenCity.
- Cross-check model: ANUGA, Geoscience Australia.

13. Status (updated Sun 11 Oct, morning)

- **New since Saturday:** offline mode, flood-safe navigation with directions and live GPS, help numbers, flood plan image, two review rounds. Sourced Chennai facts for the page: on 4 Dec 2023, 712 of Chennai's 1,814 power feeders were switched off; on 5 Dec 2023, 30% of the city's 42,747 mobile towers were down (The News Minute; The Week/PTI).
- Rough cut of the video: `review/p5/footage/rough_cut.mp4`, cut list `docs/VIDEO_CUT.md`.

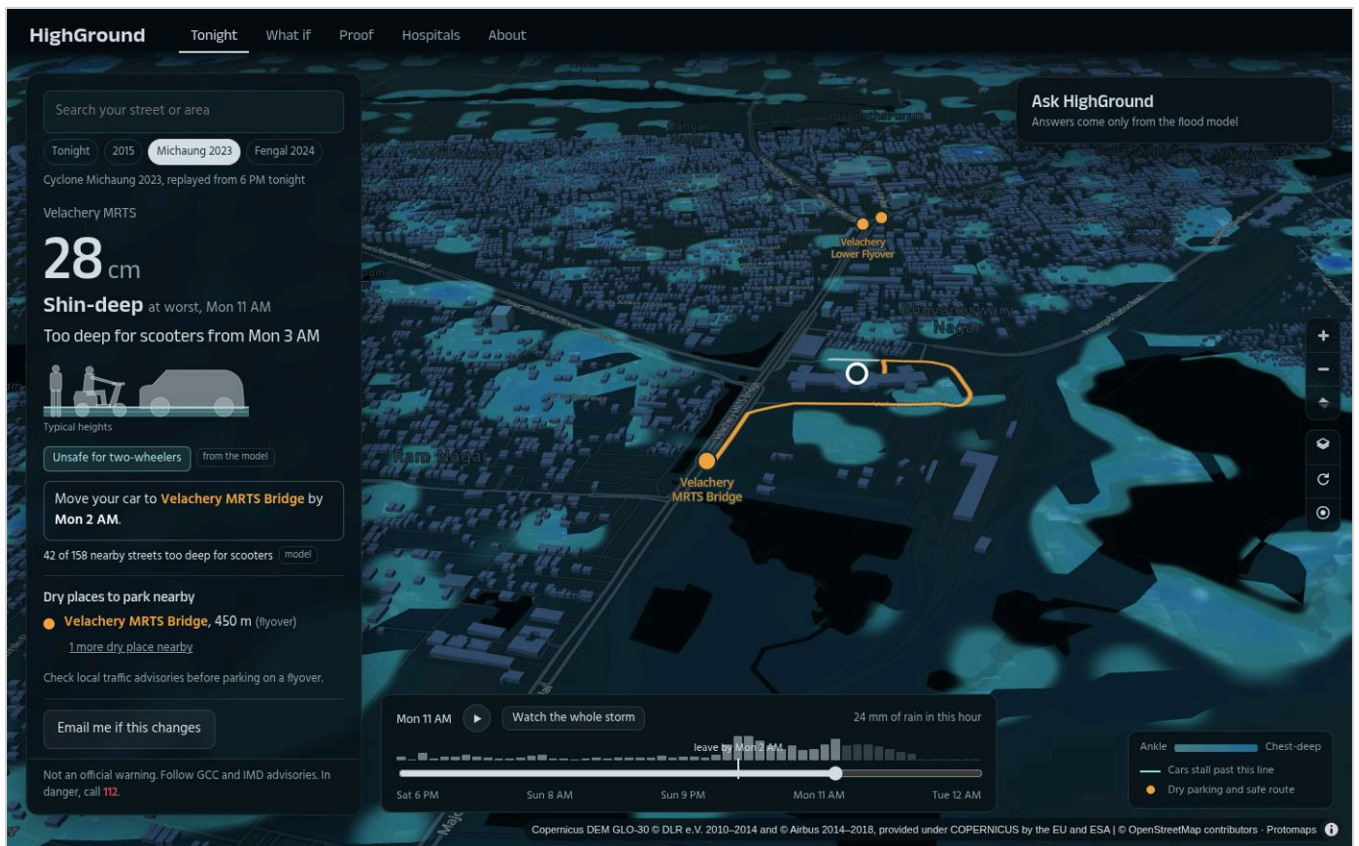
Saturday evening

- **Works locally:** the whole app (all screens, replays, time-lapse, routing, hospitals, Proof, About), the model and validation, the Lambda functions (tested end to end against mocked AWS).
- **Not yet live:** the AWS deployment (needs the team's AWS profile). Until then the assistant and email alerts show "offline". The live URL, real alert email and Bedrock assistant come after deploy.
- Run it locally: `cd web && npx vite --port 5173 (dev) or npx vite build && npx vite preview -port 4173 (fast production build)`. Demo link: `/?replay=michaung2023`. Add `&motion=on` if the device reduces motion.

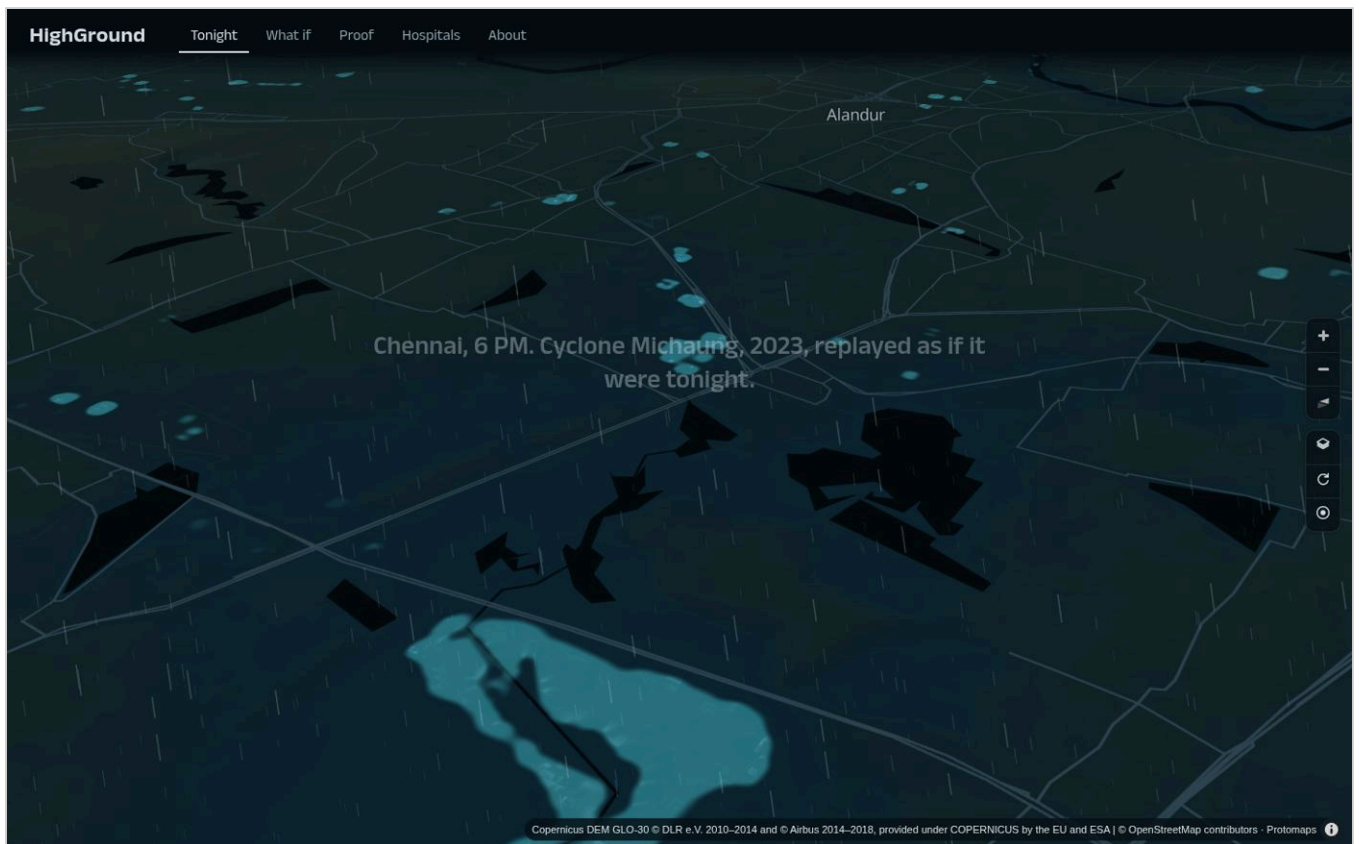
14. Glossary (for writing copy)

- **CFLOWS:** Chennai's official flood forecasting system, built for government officials.
- **Too deep for scooters:** 15 cm of water; **cars stall:** about 30 cm (the app's thresholds).
- **Replay:** a real past storm run through the model and shown as if it started at 6 PM tonight.
- **Cusecs:** cubic feet per second, the unit Chennai news uses for reservoir releases.
- **Held-out wards:** half of GCC's wards were kept aside and never used for tuning; all reported results come from them.
- **Critical success index (CSI):** of the area either model floods, the share both flood.

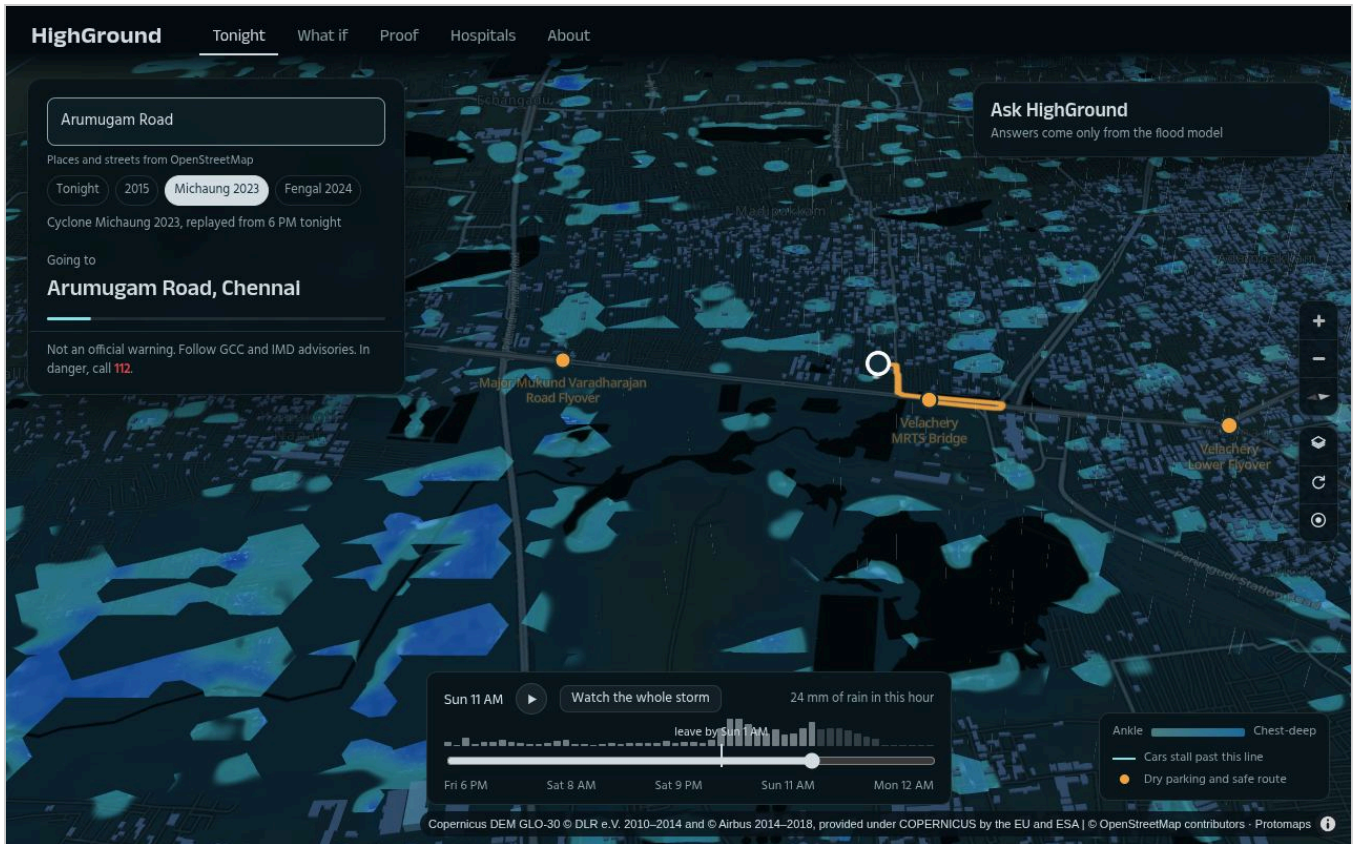
Screenshots (full-size files are in docs/landing-assets/)



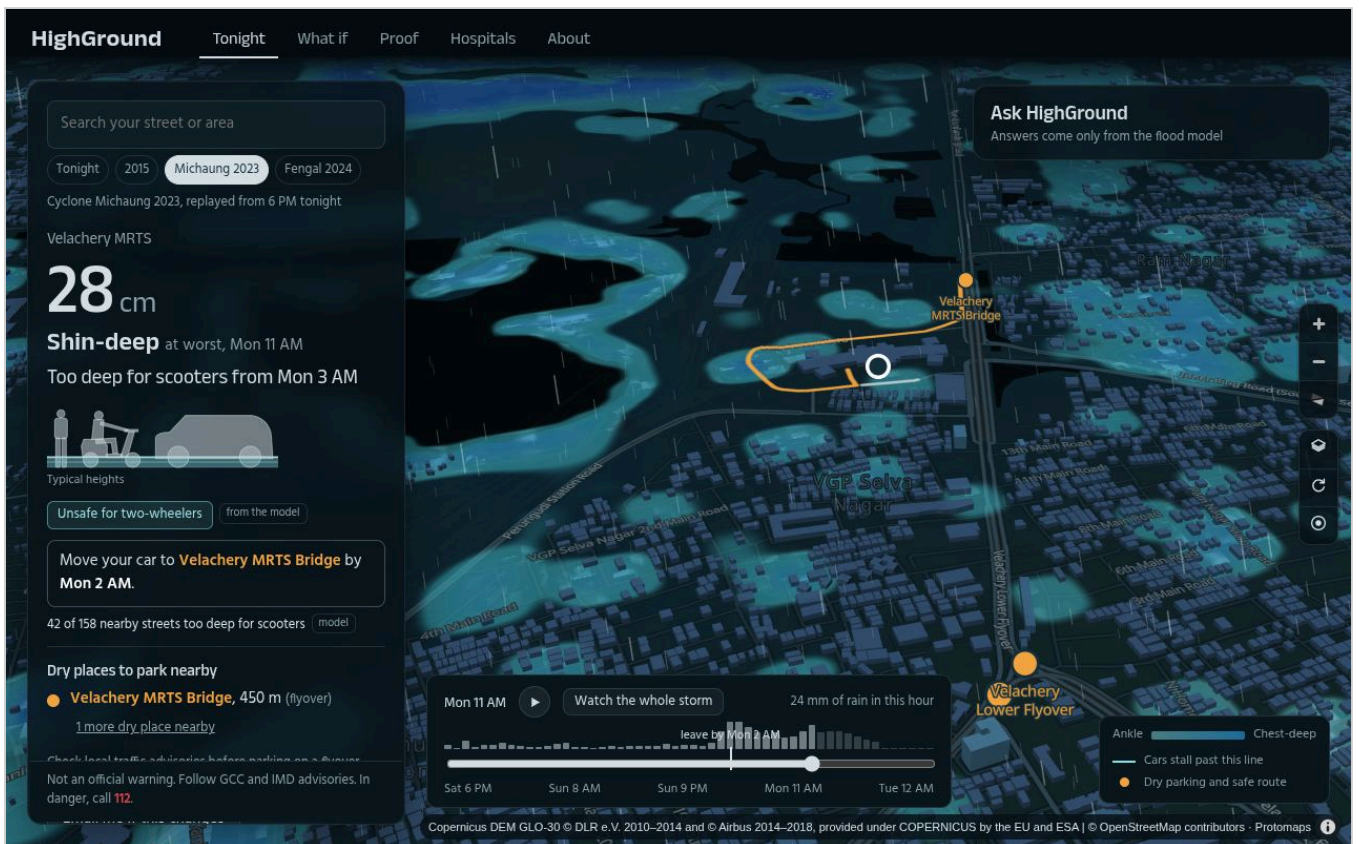
The hero: a Velachery street during Cyclone Michaung



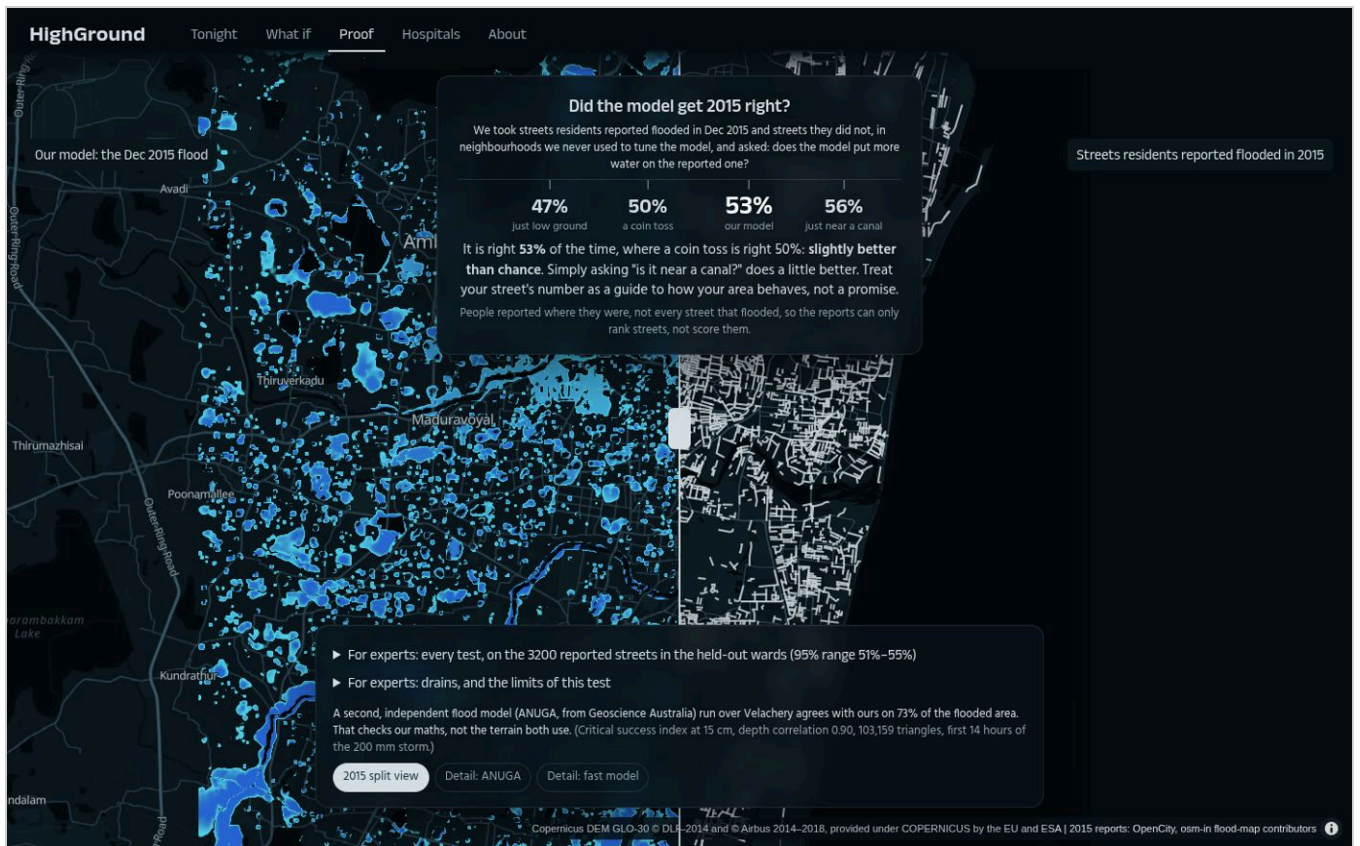
The opening title over the city



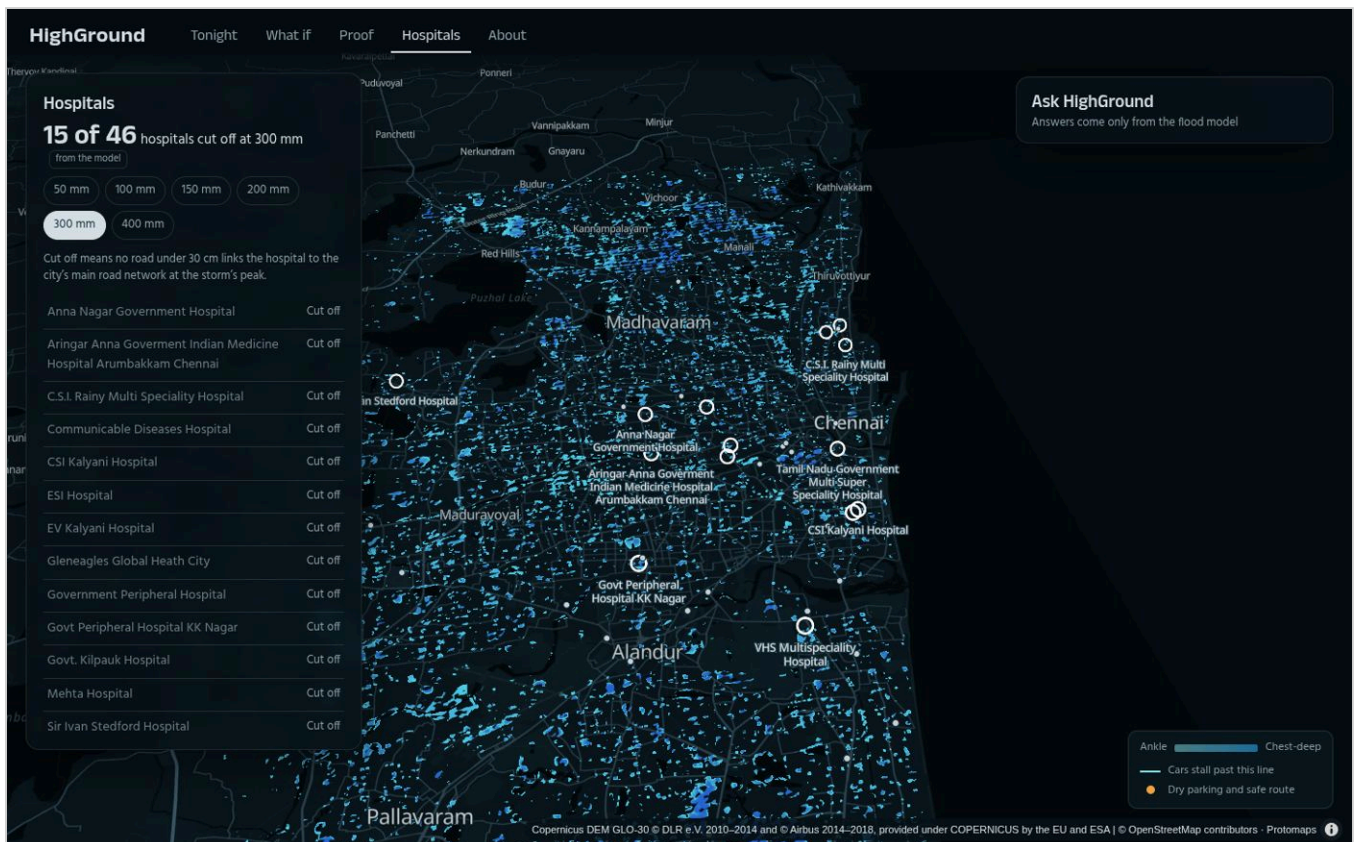
Mid-flight to a searched street



Street-level 3D view with rain



Proof: did the model get 2015 right?



Hospitals cut off at 300 mm

HighGround architecture

Offline model on the laptop; everything residents touch runs on AWS.

Offline (Python)

Open data on AWS (S3, no account)

Copernicus DEM GLO-30 s3://copernicus-dem-30m
ESA WorldCover 2021 s3://esa-worldcover
OpenStreetMap (roads, buildings, rivers, hospitals, bridges, parking)
Open-Meteo ERA5 timing, IMD storm totals

Flood model

Bare-earth terrain from the DSM, rivers burnt
Local-inertial 2D solver (numba), 30 m,
1.2 M cells, mass error < 0.01%
12 design storms + 2015, Michaung, Fengal
Drain capacity: stated assumption;
tested on held-out even wards (2015)
ANUGA cross-check for Velachery

Outputs

Depth textures (hourly), terrain-RGB tiles
Street JSON: depth, time to 15 cm
Routing graph + edge depths per storm
Dry parking, hospital reachability, proof.json

On AWS

S3 + CloudFront

Model outputs, PMTiles basemap, terrain tiles
Origin Access Control, CORS, gzip
current.json uncached (updated every 3 h)

Alerts

EventBridge Scheduler (every 3 h) / Run now
-> Lambda forecast-check: Open-Meteo, 4 points
-> nearest storms + blend -> current.json
-> risk band changed? Amazon Bedrock writes
two sentences -> Amazon SNS email
Lambda subscribe -> DynamoDB + SNS filter policy

Ask HighGround

API Gateway -> Lambda (Strands Agents SDK)
-> Claude on Amazon Bedrock
Tools read only model outputs:
get_street_risk, find_dry_parking, safe_route,
hospital_status, current_forecast, model_limits
Every number checked against tool output
Lambda geocode -> Amazon Location Service

Browser

Amplify Hosting: React app

MapLibre GL: dark basemap, 3D terrain, extruded buildings
three.js custom layer: water plane driven by depth textures, ripples, rain
Scrub hours, what-if slider, replays: texture blends, no compute
A* dry routing in the browser
Proof: swipe vs 2015 citizen reports

Resident

Types an address
Sees depth + time for their street
Dry parking + a route that avoids water
Email when the risk changes
SMS: coming soon (TRAI DLT)

sync to S3

tiles, JSON

subscribe

ask, geocode

Architecture: every AWS service